Mapping Free Functions without Compilation:

A Vertical Account of the Horizon Function Map

26 July 2026

Abstract

Horizon constructs a *function map* of a Rust repository: a navigable containment tree from repository through crates, folders, and files down to free functions, each carrying an ordered sequence of outgoing call sites. Each site resolves to exactly one definition, records a first-class `Conflict` naming every candidate, or records `Unresolved`. Recognised but out-of-scope call forms—external crates, methods, `impl` items, constructors, associated functions—are dropped from the tree entirely rather than mislabelled.

This report explains the system as a single continuous argument rather than as a catalogue of source files. It follows the transformation of source text into a map in the order the pipeline develops it: the problem that motivates a structural, compile-free analyser; the choice of parser; the data model that makes never-guessing representable; declaration-driven module discovery; fact extraction from concrete syntax; name resolution with rustc-aligned precedence; cross-crate reachability behind a dependency gate; deliberate exclusions; allowlisted macro recovery; and finally the measurement regime that distinguishes confident wrong edges from honest incompleteness. Evaluation on unfamiliar repositories shows a bimodal usefulness result: free-function pipelines yield informative maps, while method-centric and definition-body macro architectures remain thin by design.

The organising principle throughout is that **the tool never guesses**. That principle is not a slogan appended after the fact; it is the constraint from which the `Conflict` node, the drop counters, the visibility asymmetry, and the macro allowlists all follow.

Contents

Chapter 1

The Problem

1.1 What a function map is for

Software engineers routinely need a static picture of *who calls whom* across a real codebase: not a single file's outline, but a repository-scale graph that remains navigable when the project is a Cargo workspace with path dependencies, mid-edit files, and ambiguous imports. Indexing formats such as SCIP and LSIF [3, 4] already answer a related question for language servers and code hosts. They do so by invoking a full semantic analysis—typically rust-analyzer's type-aware index—and they produce a dense symbol table that includes methods, associated items, and cross-crate references into the standard library.

Horizon answers a narrower question with a stricter honesty constraint. It asks only for the free-function call structure of a Rust repository, and it insists that every edge it draws be justified by name lookup that the language itself would accept as decisive. Where the language refuses to pick a winner, the map must refuse as well. Where a call form is recognised but outside the free-function universe, the map must omit it rather than invent a label that looks like a resolution failure.

The containment spine is:

`Repository` → `Crate` → `Folder` → `File` → `Function`

Each free function carries its outgoing call sites in source order. Each call site terminates in a `CallTarget`: a resolved reference to one canonical function, a conflict naming several, or an unresolved marker. The JSON contract that emits this structure is documented separately; this report concerns the reasoning that makes the structure necessary.

1.2 Why heuristics fail

An earlier generation of the same project attempted to recover call structure with lighter machinery—directory globs for "source files," string-shaped path matching, and when ambiguity arose, a "likely" edge labelled with a confidence tag. The fixture corpus preserves the anti-example. Against a crate that rustc rejects with error `E0659` (two glob imports supplying the same name), the abandoned spike resolver emitted:

```
app.rs::run  ->  shapes.rs::get   [Likely, glob import]
```

That line is worse than silence. It asserts a unique callee where the compiler asserts that no unique callee exists. Coverage metrics that count "resolved" edges would reward the guess; a consumer navigating the map would follow a fabricated edge. The product rule that replaced this behaviour is absolute: when two globs offer the same name and no local definition or explicit import breaks the tie, the only correct emission is a `Conflict` naming both candidates.

Regex and ad-hoc path splitting fail for a second, independent reason. Rust's module system is declaration-driven. A file that sits on disk beside `lib.rs` is not part of the crate unless a `mod` declaration names it. Conversely, an inline module contributes a path segment with no file of its own, and a `#[path]` attribute can bind a tidy module name to an arbitrarily renamed file. Directory listing invents modules the compiler never sees and misses modules the compiler does.

1.3 Why parse without compiling

A natural response to the failure of heuristics is to demand a compiling project: run `cargo check`, or ask rust-analyzer for unresolved references, and map only what the toolchain accepts. That frame was considered and rejected.

The cost of a compile gate is not merely latency. It reintroduces a toolchain dependency for every analysis, refuses mid-edit repositories (precisely when a navigable map is most useful), and throws away the error tolerance that motivates choosing an industrial concrete-syntax parser in the first place. Of the diagnostic classes that arise while exercising the rules this system encodes—missing modules, private items, ambiguous globs, type mismatches— almost all parse perfectly and fail later. Syntactic validity and compilability are nearly orthogonal.

The replacement decision is: **build the tree always**. Problems become visible as first-class map outcomes—`Conflict`, `Unresolved`, or a summary counter for a deliberate drop—rather than as a refused run. Cargo remains in the pipeline, but only for crate discovery (`cargo metadata -no-deps -offline`), which is a different job from compilation.

1.4 What this buys and what it costs

Compile-free structural analysis buys three things that a SCIP-style index does not simultaneously provide at the same cost: operation on non-compiling trees; a deliberately small free-function universe that a frontend can render without understanding Rust; and an explicit representation of ambiguity that most semantic indexes collapse into "unresolved reference" or silently prefer one candidate.

It costs nearly everything that requires types. Methods (`x.foo()`), associated functions (`Type::name`), and trait resolution are permanently out of scope—not deferred phases of this tool, but a different product. Indirect calls through function pointers and closures require dataflow. Macro-expanded module trees that live only inside `macro_rules!` definition bodies remain closed. The evaluation chapter will show that these costs are not theoretical: on method-heavy repositories the map is thin by construction, even when it is correct on every free function it does index.

The remainder of this report develops the system that lives inside those constraints. Each chapter picks up where the last left off, following source text through discovery, parsing, extraction, resolution, and measurement until the shape of the design is explained rather than merely described.

Chapter 2

Design Axioms

Before any pipeline stage is examined, four axioms must be stated. They are the load-bearing decisions; later chapters are consequences.

2.1 Never guess

A call site in the indexed universe terminates in exactly one of three outcomes:

1. **Resolved** — name lookup found exactly one free-function definition. The edge holds a reference (`FunctionId`) to that canonical node.
2. **Conflict** — name lookup found two or more plausible definitions and the language does not allow picking one. The edge holds every candidate and a reason.
3. **Unresolved** — this is a free-function-shaped call, but no indexed definition matches. There are not several alternatives; there is no callee.

`Conflict` is not an error channel. It is the concrete expression of the never-guess principle: the map preserves the information that the language itself treats as decisive refusal. Collapsing conflict into unresolved would erase the distinction between "many answers" and "no answer." Picking a "likely" winner would reintroduce the spike's false edge.

2.2 The indexed universe

The never-silently-missing rule applies *within* the free-function universe the tool indexes. It does not require every syntactic `CallExpr` to appear as a site. External-crate calls, enum-variant and tuple-struct constructors, and associated functions on types are recognised and **dropped**: absent from the tree, tallied under dedicated summary counters (`external_dropped`, `constructor_dropped`, `associated_dropped`). Recording them as `Unresolved` would misdiagnose a type or an external crate as a missing free function—a different and worse failure mode than omission.

This is the intellectual centre of the design. Completeness means: nothing in-scope is silently missing. It does not mean: every parenthesis-shaped construct is forced into the graph.

2.3 Free functions only, permanently

Methods and `impl`/`trait` items are permanently out of scope for declarations and for call sites. Associated functions share that exclusion. The cost is large: of Horizon's own roughly 1,394 call sites at measurement time, about 1,053 were method calls. Resolving them needs type inference, not name lookup. A future type-aware product would be a separate system, not an incremental extension of this map.

2.4 Batch, always rebuild

The tool is a single-shot batch analyser. There is no on-disk cache, no file watcher, and no incremental state. Live updating while typing is a non-goal. The host product's policy for when to rebuild remains open; the tool itself answers one path with one map.

Chapter 3

Selecting a Parser

3.1 The criteria

Three families of Rust parsers were available:

`syn`
: The almost-universal choice for procedural macros. It produces a tidy abstract syntax tree optimised for code generation. It discards comments and refuses to parse broken input.

`tree-sitter-rust`
: An incremental, error-tolerant parser used widely in editors. It preserves a concrete tree and recovers from errors, but its Rust grammar and comment handling are a separate ecosystem from rust-analyzer's own understanding of the language.

`ra_ap_syntax`
: rust-analyzer's syntax crate. It builds a lossless concrete syntax tree (CST), preserves comments and punctuation, parses incrementally, and yields a usable tree for incomplete input.

The criteria that decided the matter were exactly the capabilities the compile-gate rejection had preserved as requirements: **error tolerance** (mid-edit code must map), **comment preservation** (doc comments are first-class map nodes), and a tree shape close enough to rust-analyzer's own that later oracle comparisons against LSIF would not fight an alien grammar.

3.2 Concrete versus abstract trees

An abstract syntax tree elides punctuation and trivia in favour of the constructs a compiler's later passes need. A concrete syntax tree retains the source's shape: attributes, doc comments, and token-tree interiors of macro calls remain addressable. For a function map that must attach documentation to the items it documents, and that must later open selected macro token trees for recovery, the CST is not an implementation detail—it is the substrate.

`syn`'s refusal to parse broken code would have made the "build always" decision incoherent. Choosing `ra_ap_syntax` and then gating on `cargo check` would have thrown away the principal reason for the choice. The two decisions reinforce each other.

3.3 What the wrapper does

The parse stage in Horizon is deliberately thin. It accepts source text and the edition string reported by cargo metadata, selects the corresponding `ra_ap_syntax::Edition` (falling back to the crate's current edition on unknown strings), and returns `SourceFile::parse(...).tree()`. Parse errors are ignored at this boundary: they live on the `Parse` value, and the tree is used

regardless. Measured extraction throughput on an early spike was roughly 1.2 MB/s single-core; later scale validation on dense trees lands around 0.8–1.1 MB/s of analysed source for the full pipeline.

3.4 What parsing does not decide

A CST tells the extractor what was written. It does not resolve names, enforce privacy, expand macros, or decide which `#[cfg]` arm is live. Those questions belong to later stages, and several of them are answered with the never-guess rule rather than with a build configuration. The parser's job ends when a tree exists.

Chapter 4

The Function Map as a Data Structure

4.1 Containment is a tree

Containment does not cycle. The emitted hierarchy is therefore a tree:

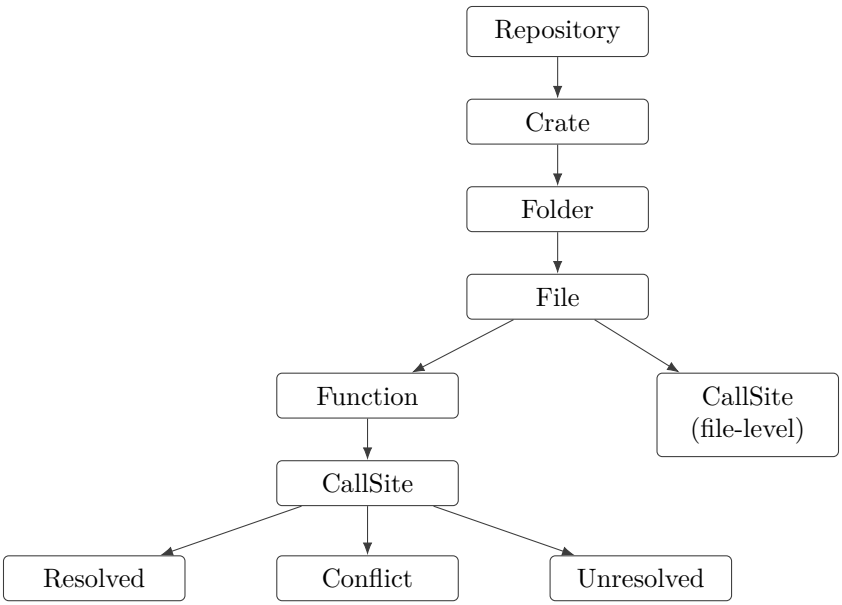

Figure 4.1: Containment hierarchy and call-target terminals. Folders nest; modules are not spine nodes. Doc-comment leaves are omitted for clarity.

Crates sit above folders because crates do not nest like directories. A repository may hold independent packages with no root manifest; a package's library and its `src/bin/*.rs` binaries are separate compilation units that share a directory. The path text `crate::...` means different things in different crates. Treating the filesystem as the spine would collapse those distinctions.

Modules are intentionally absent from the spine. The working assumption that "module equals file" is false in four ways the compiler exhibits: ordinary `mod` files, inline modules with no file, undeclared files that never compile, and `#[path]` renames. Two functions of the same name at different module depths in one file are distinguished by their recorded module paths and by their `FunctionId`s, not by inventing module nodes the folder tree cannot host.

4.2 References, not copies

A resolved call site stores a `FunctionId`—a reference to the one canonical `Function` object—never an embedded copy of that function. The same holds for conflict candidates. This is what makes recursion tractable: `countdown` calling `countdown` is a pointer back to the same node. A function called from thirty places exists once and is referenced thirty times. Expanding resolved targets by value would loop forever or explode the JSON.

Conflicts hold function references rather than file references because `#[cfg(unix)] fn open()` and `#[cfg(windows)] fn open()` can sit in the same file. Storing files would collapse them. Every function knows its parent file through containment, so the file list remains derivable.

4.3 Function identity

`FunctionId` uniquely names one definition across the repository:

Library	`{rustc_name}::{module_path}`
Binary	`{rustc_name}[bin]::{module_path}`
Collision	append `#L{line}` to every collider

The `[bin]` marker uses characters illegal in Rust identifiers and Cargo package names, so it cannot be confused with a real crate. An earlier approach that rewrote binaries to `horizon_bin` was rejected: that string looks like a package that could actually exist. The crate's `rustc_name` field always stores the real cargo target name; only the id prefix carries the disambiguator.

Internally, module paths keep a leading `crate` segment (`crate::shapes::get`). The id substitutes the crate key for that segment (`text_engine::shapes::get`). The leading segment is replaced, not prepended, so the id does not name the crate twice.

The line suffix is appended **only** when two or more definitions share the same crate-key-plus-module path. Unique paths stay free of line numbers, so ordinary ids do not churn when unrelated edits shift lines. Ids are assigned after all definitions in a crate are collected, so collisions are detected crate-wide. Because every id is prefixed by a compilation-unit key, definitions in different crates—including a package's same-named lib and bin—cannot collide inside one repository map.

4.4 Call sites on files

Calls that sit outside any free function—typically a `const` or `static` initialiser invoking a `const fn`—attach to the owning `File`, reusing the same `CallSite` type and the same resolver. Inventing a synthetic pseudo-function was rejected: it would pollute the function list and invent an identity the source does not have. Calls inside `impl`/`trait` bodies are not swept into the file-level list; methods remain out of scope.

4.5 Summary counters

The repository carries a `MapSummary`: counts of conflicts, unresolved sites, and the three drop categories. Resolved edges are not counted there. The counters exist so map health is visible without walking the tree, and so "excluded by design" is never conflated with "could not resolve."

Chapter 5

Discovering Compilation Units

5.1 Cargo metadata as a read-only oracle

Crate discovery runs `cargo metadata -no-deps -format-version 1 -offline` per manifest, retrying without `-offline` if the lockfile or index is unavailable, but never dropping `-no-deps`. Full metadata can create or update `Cargo.lock` in a repository the tool does not own—forbidden for a read-only analyser. The invocation performs no compilation; on Horizon itself it was measured at a few hundred milliseconds.

`-no-deps` lists only workspace members. Path dependencies outside that set are found by recursing into each dependency's `path` directory and running metadata there. Registry and git dependencies are never walked.

5.2 One node per compilation unit

Discovery emits one `Crate` node per mapped target, not per package. Library-like kinds (`lib`, `rlib`, `dylib`, `cdylib`, `staticlib`, `proc-macro`) and binaries are included. Examples, integration tests, benches, and `build.rs` are deliberately excluded: they are secondary surfaces that inflate maps and hit the `[dev-dependencies]` blind spot.

`proc-macro` is included explicitly because cargo metadata reports it as its own kind. Omitting it silently dropped crates such as `serde_derive`—ordinary free functions inside procedural-macro crates would vanish from the map.

Same-package binaries receive an implicit path dependency on their package library, matching Cargo. Manifest rename aliases (`alias = { package = "real-name", path = "..." }`) are recorded: the import rustc name (alias when present) is what code and the dependency gate use; the package name identifies the path edge.

5.3 Fixtures must not pollute self-maps

When analysing the Horizon repository itself, discovery skips `tests/fixtures/**`. Those crates are deliberately broken or ambiguous specimens. Mapping them as ordinary source would corrupt every self-test. Nested fixture packages also declare an empty `[workspace]` table so Cargo does not claim them when checked via `-manifest-path`.

5.4 Consequence for resolution

Once crates are spine nodes with declared dependency lists, the dependency graph becomes a hard constraint on resolution. If crate A does not depend on crate B, no name in A may resolve

into B—a far more reliable filter than name uniqueness across a repository. Chapter 9 returns to this gate.

Chapter 6

Following the Module Tree

6.1 Declaration-driven discovery

From each target's root file, the walker follows `mod` declarations transitively. It never globs the directory. The four compiler-verified divergences between modules and files are the proof that globbing is wrong:

Case	Module	File
Ordinary	`text`	`text.rs`
Inline module	`text::case`	none — inside `text.rs`
Undeclared	none	`orphan.rs` — never compiled
`#[path]`	`tidy_name`	`renamed_on_disk.rs`
Crate root	the root itself	`lib.rs` — no path segment

Table 6.1: Module/file divergences that forbid directory-glob discovery.

The fixture `phase2-modules` places garbage in an undeclared `orphan.rs`:

```
//! Undeclared garbage --- must never appear in the function map.
this is not rust!!!
fn should_not_be_indexed() {
    nonsense();
}
```

A file containing pure syntactic garbage compiles fine as long as no `mod` declaration names it. Globbing would index it; the declaration walk never opens it.

The `path-dependency` fixture binds a tidy name to a renamed file:

```
#[path = "renamed_on_disk.rs"]
pub mod tidy_name;
```

The module path is `crate::tidy_name`, not the on-disk stem. Calls and ids must use the module name the language sees.

6.2 Inline modules and nesting

An inline `mod name { ... }` extends the module path and visibility tables but creates no `File` node. A subtle consequence, verified in fixtures: a file `selfdecl.rs` declared as `mod selfdecl` that itself contains `pub mod selfdecl { ... }` *nests* rather than merging. The inner function's path is `crate::selfdecl::selfdecl::inner_fn`.

6.3 First declaration wins

Repeated declarations of the same module path terminate that branch: the first declaration wins. The practical limitation is cfg-twinned modules with different `#[path]` targets—only the first file is kept. One module path maps to one file. Cycles are handled by the same seen-set. Missing module files are reported and skipped without panicking.

6.4 Item-macro module recovery (preview)

`mod` declarations hidden inside allowlisted item macros (`cfg_if!`, Tokio-style names starting `cfg_`) are recovered by re-parsing the invocation token tree, depth-bounded at 8, with all `cfg_if!` branches unioned. The tool does not choose a build configuration—the same spirit as keeping `#[cfg]`-duplicated functions. Chapter 11 develops the full recovery argument, including why serde's `crate_root!()` pattern stays closed.

6.5 Folders from walked files only

After extraction and resolution, each compilation unit builds its folder tree from the files the module walk found, independently of every other unit. A library and its `src/bin/` binaries sharing a directory do not interleave files. Undeclared neighbours never appear.

Chapter 7

Extracting Facts from Concrete Syntax

7.1 What extraction collects

For each discovered file, at the crate's real edition, extraction walks the CST and collects:

- free-function definitions (with visibility), including nested free functions;
- local type names—structs, enums with variants, traits, type aliases—used later to classify constructors and associated functions;
- the import table from `use`/`pub use` trees and `extern crate` renames, flattened from the AST rather than by string splitting;
- path-form call sites, owned either by an enclosing free function or by the file;
- doc comments: outer forms on functions, inner forms on files.

Methods and anything inside `impl`/`trait` items are skipped at extraction time. They never become pending call sites, so they cannot be mislabelled as unresolved free functions later.

7.2 Import tables

Supported forms include plain paths, brace lists, nested braces, aliases, `self` in a brace list (binds the module), globs, re-exports, renamed re-exports, glob re-exports, relative roots (`self::`, `super::`, `crate::`), and `pub extern crate dep as name`. Discard imports (`as _`) contribute no binding. External roots such as `std::fs` are recorded as external so later calls through the binding can be dropped rather than unresolved.

A known limitation is recorded rather than hidden: a `use` inside a function body is attributed to the enclosing module with `scope_widened = true`. Resolution therefore treats it as module-wide, which is wider than rustc. The flag exists so the limitation is never silent; as of this writing the resolver does not special-case the flag—the widening is unconditional for such imports.

7.3 Doc comments

`ra_ap_syntax` keeps outer docs and attributes as children of the item, so `///` docs followed by `#[inline]` still attach to the function. Consecutive pieces of the same kind on one owner are joined with newlines into a single `DocComment`, matching rustdoc's one-logical- comment rule. Ordinary `//` comments never appear. Whitespace stripping removes exactly one leading ASCII space after the marker and preserves further indentation for fenced code blocks.

7.4 Pass ordering

Every discovered crate is extracted before any crate is resolved. Cross-crate edges need callees to exist. Function ids are assigned crate-wide after each crate's definitions are known, then pending call owners are remapped to those ids. The shared `pipeline` module constructs the extract → resolve-index path used by both the CLI and the correctness harness, so measurements cannot drift from production.

Chapter 8

Name Resolution as a Calculus

8.1 The intellectual centre: Conflict

Consider the minimal fixture that exists solely to trigger rustc `E0659`:

```
use crate::shapes::*;
use crate::text::*;

pub fn run() -> usize {
    get(3)
}
```

Both sibling modules define `get`. There is no explicit import and no local definition. rustc rejects the crate. The abandoned spike guessed `shapes::get`. Horizon emits a `Conflict` naming `glob_ambiguity::shapes::get` and `glob_ambiguity::text::get`.

The twin fixture adds one line:

```
use crate::text::get;
```

rustc accepts the crate; the explicit import outranks the globs; `get(3)` resolves uniquely to `text::get`. The map must emit a single resolved edge—not a conflict, and not a coin-flip.

These two crates are the clearest statement of the design. Ambiguity is not a degraded form of success. It is a different terminal.

8.2 Precedence

Within-crate unqualified lookup follows rustc, verified against the glob fixtures:

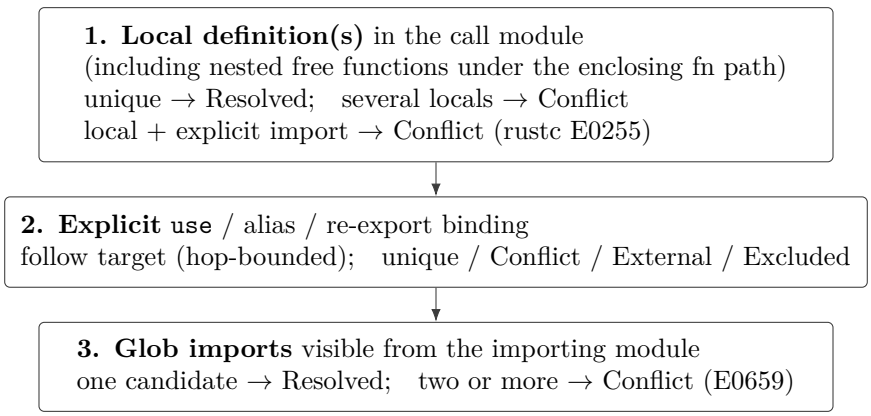

Figure 8.1: Unqualified name-resolution precedence. The tool never picks a winner among equal-ranked candidates.

Qualified paths navigate `crate::`, `self::`, `super::` (including multi-`super`), then module segments and import bindings. Facade re-exports are followed with a shared hop bound of 32 (`REEXPORT_HOP_LIMIT`). Exhausting the bound yields `Unresolved`; the cross-crate entry path includes the limit in the reason string so a re-export cycle cannot hang the tool.

8.3 Visibility asymmetry

Three distinct policies apply. They look inconsistent until each is justified:

Direct within-crate edges.
Visibility is *not* filtered. A path the author wrote still appears even when rustc would emit `E0603`. Mid-edit code must stay mappable. The never-guess rule forbids picking among candidates; it does not require pretending a written call does not exist.

Glob candidate sets.
Visibility *is* filtered. A glob only brings in names visible from the importing module through the globbed module. Including private items would manufacture `E0659` conflicts rustc would never report—false conflicts, the dual of false resolutions.

Cross-crate reachability.
Visibility *is* enforced: a `pub` item behind an all-`pub` module chain, or a `pub` facade that exposes the item without naming private intermediates. `pub(crate)` stops at the crate boundary.

Keep the asymmetry deliberate. Within-crate direct edges map what the author wrote. Cross-crate edges map what is actually nameable from outside. Glob filtering protects the conflict predicate from pollution.

8.4 Cfg twins

```
#[cfg(unix)]
fn open() {}

#[cfg(windows)]
fn open() {}
```

Both definitions appear in the syntax tree. Function ids receive `#L{line}` suffixes. A call to `open()` becomes a `Conflict` naming every candidate. The tool does not choose a build configuration. That is the never-guess outcome, not a silent pick—and on real corpora it is a common, honest source of map noise.

8.5 Unresolved is not a dustbin

`Unresolved` means: this is a free-function call we genuinely could not resolve. Typical causes include a typo, a name that exists only behind an unopened macro, a path into a workspace sibling that is not a declared dependency, or—on real corpora—test-only externals that discovery never listed because `[dev-dependencies]` are omitted. It must not be used for constructors, associated functions, or external crates. Those have their own exits from the resolver.

Chapter 9

Crossing Crate Boundaries

9.1 The dependency gate

Crate A may resolve into crate B only when A declares B as a path dependency. Workspace membership alone is not enough. The `workspace-dep-gate` fixture places the same `pub fn shared` in `dep_a` and `dep_b`; the consumer depends only on `dep_a`. Naming `dep_b::shared` must remain `Unresolved`—never a wrong edge into the undeclared sibling.

This gate is the payoff for elevating crates to spine nodes (Chapter 5). Name uniqueness across a repository is a weak filter; the declared dependency graph is a hard one.

9.2 Cross-crate visibility

Once the gate opens, reachability still requires:

1. every *named* module segment from the foreign crate root to be `pub`;
2. the target item itself to be `pub`;
3. or a `pub` facade that exposes the item under a public name, including `pub use private_mod::item` (Rust-correct: the re-export name is public even when the module is not).

`pub(crate) use` is invisible from outside. A path that names a private foreign module (for example `text_engine::secret::hidden`) remains unreachable even if the function inside is `pub`.

9.3 Facade following

Published Rust libraries conventionally present a flat crate-root API and treat internal crates as private. Leaving facade paths unresolved was the largest remaining resolution gap on multi-crate corpora. Horizon follows:

- `pub use eng::upper`
- renamed `pub use eng::tokenize as split`
- `pub use eng::*`
- `pub extern crate eng as name` (ripgrep-style barrels)
- multi-crate chains (facade → mid → leaf)

Following a hop into a third crate that the *foreign* crate declares does not require the consumer to declare that third crate. The consumer's *initial* entry still needs the dependency edge. Two foreign glob re-exports offering the same name produce a `Conflict`, never a pick. All re-export walks share the hop bound of 32.

Scale validation after facade following recovered ripgrep edges such as `grep::cli::hostname` resolving to the defining function in `grep_cli`, and rust-analyzer barrel paths such as `hir::db::file_item_tree`

resolving into `hir_def`. Hand spot-checks of the newly resolved facade sample found no false positives.

9.4 Consumer-side globs remain closed

`use dep::*` in the *calling* crate is not specially expanded. Foreign crates' own `pub use dep::*` facades *are* followed. The distinction matches how published APIs are actually structured: the facade author chooses the public surface; a consumer glob is a different, wider request that would require importing another crate's entire public namespace into the local conflict calculus.

Chapter 10

Deliberate Absences

Each exclusion below is a design decision with a recorded cost, not a gap waiting to be filled inside this tool.

10.1 Methods and associated functions

Method declarations and `x.foo()` call sites are permanently out of scope. Associated functions (`Vec::new`, `Type::assoc`) share the exclusion and are counted in `associated_dropped`. Both are `impl` items. Resolving them needs the type of the receiver or the type segment, which is type inference. Even a future type-aware tool would still face receivers behind generics and trait objects.

Classification of associated-function forms prefers collected type facts, prelude names, and primitives; when facts are absent it falls back to a strict UpperCamelCase heuristic (ASCII uppercase start, alphanumeric only, at least one lowercase—not `ALL_CAPS`). Segments that fail the heuristic surface as `Unresolved` rather than silent drops. Modules present in the module tree are navigated as modules regardless of capitalisation; UpperCamelCase modules *absent* from the tree can still be mis-dropped—a documented residual risk.

10.2 Constructors

Enum variants and tuple-struct constructors are syntactically `CallExpr` in `ra_ap_syntax` but construct a value rather than invoke a free function. They are absent from the tree and counted in `constructor_dropped`. Recording `Ok(x)` as unresolved would mis-diagnose a prelude constructor as a missing function.

10.3 External calls

Calls into `std`/`core`/`alloc`/`proc_macro`, or into a Cargo dependency whose kind is external, are deliberately dropped and counted in `external_dropped`. Path dependencies are not dropped by this rule. Someone reading the never-silently-missing rule literally might try to "restore" external calls as unresolved sites. Do not: external callees are outside the indexed universe. A function whose body only calls `std` appears as a childless leaf; that is intentional.

10.4 Indirect calls and functions as values

A call through a parameter or closure binding needs dataflow. `apply(&n, normalize)` records the call to `apply` but not the reference to `normalize`. Both relationships are real and both are absent; the second vanishes completely from the map.

10.5 Secondary Cargo surfaces

Examples, tests, benches, and `build.rs` are not discovered. `[dev-dependencies]` are omitted from the dependency list used for resolution. Scale validation shows the cost: test-only externals often surface as `Unresolved` rather than `external_dropped`. Including them as external roots (not walked) remains an open product decision.

10.6 SCIP as production indexer

`rust-analyzer scip` produces a full semantic index with real name resolution and type inference—measured at 73 s / 2.1 GB on ruff with roughly 81% of references resolved, including methods. The hand-built approach resolves a smaller fraction of path-form calls with methods excluded. The decision to build a custom free-function map anyway trades coverage for compile-free operation, an explicit conflict terminal, doc comments as first-class nodes, and independence from a rust-analyzer component in the production path. LSIF from rust-analyzer is used only as a correctness oracle on Horizon itself, not as the production indexer.

Chapter 11

Recovering What Macros Hide

11.1 Why token trees make calls invisible

The parser leaves macro arguments as unstructured token trees. The call `mean(v)` inside `format!("{}", mean(v))` is invisible to a normal `CallExpr` walk. Measured on Horizon, this hid on the order of tens of path-form calls against a few hundred visible ones—enough to matter, not enough to justify opening every macro.

11.2 Allowlist and reparse

Recovery opens only an allowlist of std/core macros whose arguments are expression positions: `format!`, `print!`/`println!`/ `eprint!`/`eprintln!`, `write!`/`writeln!`, `assert!` family, `debug_assert*` family, `vec!`, `dbg!`, `panic!`, `unreachable!`, `unimplemented!`, `todo!`. For each, the token-tree interior is re-parsed as Rust (parenthesised arguments become arguments of a synthetic callee; brackets become an array literal) and path-form `CallExpr`s are collected with ranges mapped back into the original file. Nested allowlisted macros found after reparse are opened recursively, depth-bounded. Recovered sites resolve through the normal path and set `CallSite.from_macro` (omitted from JSON when false).

A span-fidelity check requires the mapped byte range to match the original token-tree text, rejecting reparse phantoms. Deduplication against ordinary CST sites prevents double counting.

11.3 Look-alikes that must stay closed

The same token shape appears in places that are not calls:

```
// Genuine calls hidden in token trees (must be recovered).
let _ = format!("{}", mean(v));
println!("{}", width_of(s));

// Traps --- must stay absent from the map.
let _ = matches!(Some(1), Some(_y));   // pattern, not a call
let _ = stringify!(helper());          // tokens, not evaluation
let _ = vec![Foo(1), Foo(2)];          // constructors
identity_call!(mean(v))                // macro_rules / user macro
```

`matches!`, `stringify!`, `concat!`, `include*!`, `cfg!`, `quote!`, `macro_rules!` definition bodies, attribute and derive token trees, and all user-defined macros stay opaque. Prefer a miss over a fabricated edge. Tuple-struct constructors recovered inside allowlisted macros still hit the classifier and are dropped as constructors.

11.4 Item-macro module recovery

Item-pasting macros hide `mod` declarations the same way. The allowlist is `cfg_if!` and names starting with `cfg_` (Tokio-style `cfg_fs!`, ...). All `cfg_if!` branches are unioned. Nested allowlisted macros open up to depth 8. Missing files on a branch are skipped without panicking.

Not recovered—serde `crate_root!()`. The real modules live in a `macro_rules!` *definition* body; the invocation is an empty `crate_root!()`. Expanding definition bodies is a different and riskier problem than re-parsing an invocation's tokens. Prefer a missing subtree over a fabricated module tree. Scale validation confirms the consequence: after proc-macro discovery, `serde_derive` appears, but serde's main library remains thin.

Chapter 12

Measuring Correctness

12.1 Coverage is not correctness

A confidently wrong `Resolved` edge is worse than an honest `Conflict`. Coverage metrics that reward resolution rate incentivise exactly the spike's "likely" behaviour. Correctness measurement therefore asks a different question: among edges the tool claims to have resolved, how often is that claim false?

12.2 Three complementary oracles

Hand-annotated fixtures.
Primary standing check. `expected-edges.json` files encode required resolved targets, conflicts, unresolved sites, and required absences. They cover non-compiling crates that SCIP/LSIF cannot index.

Adversarial fixture.
Directly probes confident wrong edges: same name at several module depths, local-versus-glob, rename chains, cfg twins, module/function name collision, nested-inline shadowing.

rust-analyzer LSIF on Horizon itself.
Compiler-grade name resolution over real code, filtered to free-function monikers. SCIP is available but LSIF is JSON and needs no extra crate for comparison.

The harness never changes the resolver's answers; it only compares them. It shares the extract → resolve-index pipeline with the CLI.

12.3 Snapshot results and what they establish

On the standing harness (26 July 2026 snapshot): 59 / 59 fixture oracle edges passed with **zero false positives**; every required conflict stayed a conflict; no case resolved to a guessed winner. On an atomically generated self-map + LSIF pair: 411 resolved edges compared, 411 agreed with a same-name free-function moniker, **zero false positives**, including a separate cohort of 37 macro-recovered sites.

12.4 What zero false positives does not establish

Treat the result with proper scepticism.

1. Everything measured at LSIF grade is Horizon's own small, clean source tree plus purpose-built fixtures. That is not evidence about large, messy corpora.

2. Cross-crate precision beyond multi-crate fixtures lacks a foreign `FunctionId` LSIF filter; newly resolved facade edges on large corpora still need hand spot-checks.
3. Visibility edges that rustc would reject but Horizon still draws (by design, for direct within-crate paths) are not false positives under the product rule, and are not compared to rustc accept/reject.
4. Nested-function recursion by unqualified name can surface as `Unresolved`; product code currently avoids the pattern; no dedicated fixture yet.
5. Drop categorisation (external vs constructor vs associated) has been sampled, not exhaustively proven on unfamiliar code.

High confidence is warranted for the claim "Horizon is not systematically guessing on free functions in the cases we can check." Lower confidence attaches to "every drop is perfectly categorised" and to behaviour on large external corpora.

Chapter 13

Evaluation at Scale

13.1 Corpus and method

Correctness fixtures cannot speak to usefulness. Scale validation runs the release binary on unfamiliar repositories: Cellular Automata (free-function heavy), serde (macro-assembled), ripgrep (classic free-function workspace), bat, tokio (method-heavy, `cfg_*` modules), and rust-analyzer (large). Throughput is analysed-bytes per wall-clock second; usefulness is judged qualitatively against what a free-function map can possibly show.

13.2 Throughput

Dense trees land around **0.8–1.1 MB/s** of analysed source. Linear extrapolation from rust-analyzer scale (∼1.5k disk files, ∼21 s) suggests a ∼50,000-file dense repository is a batch job of roughly **10–20 minutes**, with peak memory in the low-gigabyte range—not an interactive refresh. No panics, hangs, or runaway memory were observed up to that scale in the measured pass.

13.3 The bimodal usefulness finding

Codebase style	**Map quality**
Free-function pipelines (ripgrep, Cellular Automata)	Useful: recognisable pipelines, modest unresolved rates among kept sites
Facade crates + path re-exports	Improved after cross-crate facade following
Allowlisted item-macro trees (`cfg_if!`, Tokio `cfg_*`)	Recovered (Tokio 114→492 map files)
Definition-body expanders (serde `crate_root!`)	Thin: proc-macro crate visible; main library modules stay closed
Method / `impl`-centric (tokio runtime)	Partial: free helpers visible; product logic missing by design

Table 13.1: Qualitative usefulness by codebase style.

On ripgrep, the map shows thousands of resolved free-function edges. On tokio, after `cfg_*` recovery, hundreds of free functions appear—but the runtime's real structure still lives in methods, which are never counted even in `associated_dropped`. On serde, discovering `serde_derive` lifts crate and function counts while the main library's definition-body modules remain invisible. The honest verdict: **the tool serves free-function pipelines well and method-heavy architectures only partially, by permanent design.**

13.4 Robustness observations

Cfg-blind duplicate definitions produce honest `Conflict` noise throughout ripgrep and rust-analyzer. Dev-dependency blindness inflates `Unresolved`. Non-package `Cargo.toml` fixtures (bat's syntax tests) produce skip warnings and continuation. Raw-identifier module paths can miss a file. First-wins on cfg-twinned `#[path]` modules remains a real limitation. None of these emptied a corpus member or crashed the run.

Chapter 14

Limitations and Future Work

14.1 Accepted limitations

- Modules are not spine nodes; inline modules vanish from the folder tree and survive only as path segments on functions and files.
- Cfg-twinned `#[path]` modules: first declaration wins.
- Function-body `use` is widened to module scope.
- Re-export chains stop at 32 hops.
- Feature-gated and target-specific dependencies are undecidable without choosing a feature set and target.
- Macro recovery outside the allowlists stays closed, including definition-body expanders.
- Consumer-side cross-crate globs are not expanded.
- Indirect calls and functions-as-values remain absent.

14.2 Open questions

Update trigger and host-product caching policy remain unspecified. Pipeline parallelism is the obvious performance lever (Pass 1 must still finish for every crate before resolve). Broader macro recovery would need its own false-positive measurement. Cross-crate LSIF breadth, UpperCamelCase module mis-drops, nested free-function unqualified recursion, and treating dev-dependencies as external roots are recorded as open. Multi-language support and type-aware method resolution belong to other products.

14.3 Implementation notes noticed in reading

A vertical reading of the whole system surfaces a few incoherences between documentation and code that do not overturn the design but should be known:

1. **Stage numbering drift.** `lib.rs` numbers stages as discover → modules → parse → extract → resolve; some module banners shift those numbers by one. Behaviour is consistent; labels are not.
2. **Hop-limit reason strings.** Documentation states that exhausting the re-export hop bound yields `Unresolved` with a reason naming the limit. The cross-crate entry path does so; several within-crate helpers return a bare unresolved outcome without mentioning the limit.
3. **`scope_widened`.** Extraction records the flag; resolution does not branch on it—widening is unconditional, which matches the documented limitation but makes the flag observational.
4. An unused visibility computation exists in one within-crate lookup helper (a boolean is computed and discarded); the effective policy is the documented asymmetry, implemented

by narrower checks elsewhere.

At the time of writing, a concurrent analysis document on unresolved sites (`docs/unresolved-analysis.md`) had not yet appeared; its findings are therefore not folded into this report.

Chapter 15

Conclusion

Horizon is best understood as a vertical slice through a narrowing of Rust: from repository to free-function call edge, without compilation, without guessing, and without pretending that methods are free functions. The `Conflict` node is the design's centre of gravity. Around it arrange the parser choice, the declaration-driven module walk, the three-way visibility policy, the dependency gate, the drop counters, and the macro allowlists—each a way of refusing to invent structure the language or the tool's universe does not support.

The measurement regime shows that this refusal is operationally attainable on the corpora that can be checked: zero false positives on fixture oracles and on LSIF-comparable self-map edges, including macro-recovered sites. The scale regime shows that refusal has a usefulness cost: free-function pipelines light up; method-centric and definition-body-macro architectures stay dark where the program actually lives.

That bimodal result is not a defect in the implementation. It is the design, stated plainly.

Appendix A

Fixture Corpus

Each fixture under `tests/fixtures/` isolates one resolution hazard. They are specimen inputs, not part of the Horizon build; several are intentionally non-compiling. The standing correctness harness consumes those that carry `expected-edges.json`.

Fixture	Hazard
`phase1-single-file`	Same-file resolution, recursion, cfg twins
`phase2-modules`	`mod` walk, `super`, orphan file, file-level call
`glob-ambiguity` / `glob-resolved`	E0659 vs explicit-import win
`path-dependency`	Path deps, visibility, `#[path]`, selfdecl nesting
`workspace-dep-gate`	Undeclared workspace sibling must not resolve
`cross-crate-facade`	Facade re-exports, glob conflict, dependency gate
`adversarial-resolution`	False-positive traps
`macro-hidden-calls`	Expression-macro recovery vs look-alikes
`macro-modules`	Item-macro `mod` recovery vs `stringify!`
`exclude-non-functions`	Drops vs genuine unresolved
`renamed-path-dep`	Cargo manifest rename aliases
`proc-macro-crate`	`proc-macro` target kind discovery
`lib-and-bin`	`[bin]` FunctionId disambiguation
`doc-comments`	Outer/inner/block/`#[doc]` attachment

Table A.1: Primary fixtures and the hazards they isolate.

Bibliography

[1] The Rust Project Developers, *The Rust Reference*. `https://doc.rust-lang.org/reference/`.

[2] rust-analyzer Contributors, *rust-analyzer* (including the `ra_ap_syntax` crate and LSIF/SCIP export). `https://rust-analyzer.github.io/`.

[3] Sourcegraph, *SCIP: Source Code Indexing Protocol*. `https://github.com/sourcegraph/scip`.

[4] Microsoft / LSIF community, *Language Server Index Format (LSIF)*. `https://microsoft.github.io/language-server-protocol/specifications/lsif/0.6.0/specification/`.

[5] The Cargo Team, *The Cargo Book* (metadata, workspaces, dependency renaming). `https://doc.rust-lang.org/cargo/`.